

```
In [28]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px

# Load dataset
df = pd.read_excel(r'C:\Users\Vinay Kumar\Downloads\naukri data.xlsx')

# Quick overview
df.head()
df.info()
df.describe(include="all")
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 25114 entries, 0 to 25113
```

```
Data columns (total 16 columns):
```

#	Column	Non-Null Count	Dtype
0	Job Posting ID	25114 non-null	int64
1	Job Posting Date	25114 non-null	datetime64[ns]
2	Job Title	25114 non-null	object
3	Job Title Full	25114 non-null	object
4	Job Position Type	25114 non-null	object
5	Job Position Level	25114 non-null	object
6	Years of Experience	25114 non-null	int64
7	Job Skills	22904 non-null	object
8	Job Location	25114 non-null	object
9	Minimum Pay	1812 non-null	float64
10	Maximum Pay	1812 non-null	float64
11	Pay Rate	1812 non-null	object
12	Number of Applicants	17529 non-null	float64
13	Company Name	25114 non-null	object
14	Company Industry	25114 non-null	object
15	Unnamed: 15	1155 non-null	float64

```
dtypes: datetime64[ns](1), float64(4), int64(2), object(9)
```

```
memory usage: 3.1+ MB
```

Out[28]:

	Job Posting ID	Job Posting Date	Job Title	Job Title Full	Job Position Type	Job Position Level	Years of Experience
count	2.511400e+04	25114	25114	25114	25114	25114	25114.0
unique	NaN	NaN	53	4394	6	7	
top	NaN	NaN	Software Engineer	Software Engineer	Full-time	Mid-Senior level	
freq	NaN	NaN	8112	2147	19967	13577	
mean	2.688359e+09	2020-01-05 16:48:35.091184128	NaN	NaN	NaN	NaN	4.5
min	1.573681e+09	2017-01-01 00:00:00	NaN	NaN	NaN	NaN	0.0
25%	2.678769e+09	2018-11-26 00:00:00	NaN	NaN	NaN	NaN	3.0
50%	2.696126e+09	2020-05-29 00:00:00	NaN	NaN	NaN	NaN	4.0
75%	2.721763e+09	2021-03-11 00:00:00	NaN	NaN	NaN	NaN	6.0
max	2.797780e+09	2021-12-31 00:00:00	NaN	NaN	NaN	NaN	25.0
std	6.212277e+07	NaN	NaN	NaN	NaN	NaN	2.2

In [29]:

```
# Rename columns for clarity and consistency
df = df.rename(columns={
    "Job Posting ID": "job_id",
    "Job Posting Date": "posting_date",
    "Job Title": "job_title",
    "Job Title Full": "job_title_full",
    "Job Position Type": "position_type",
    "Job Position Level": "position_level",
    "Years of Experience": "years_experience",
    "Job Skills": "skills",
    "Job Location": "location",
    "Minimum Pay": "min_pay",
    "Maximum Pay": "max_pay",
    "Pay Rate": "pay_rate",
    "Number of Applicants": "Applicants",
    "Company Name": "company_name",
    "Company Industry": "industry",
    "Unnamed: 15": "misc"
})

# Verify new column names
print(df.columns.tolist())
```

```
['job_id', 'posting_date', 'job_title', 'job_title_full', 'position_type', 'position_level', 'years_experience', 'skills', 'location', 'min_pay', 'max_pay', 'pay_rate', 'Applicants', 'company_name', 'industry', 'misc']
```

```
In [26]: # Check missing values
missing_summary = df.isnull().sum().reset_index()
missing_summary.columns = ["Column", "Missing_Count"]

# Focus on salary columns
min_nulls = df['min_pay'].isnull().sum()
max_nulls = df['max_pay'].isnull().sum()

print(f"Missing Min_Pay: {min_nulls}, Missing Max_Pay: {max_nulls}")

# Fill or drop strategy
df['min_pay'] = df['min_pay'].fillna(df['min_pay'].median())
df['max_pay'] = df['max_pay'].fillna(df['max_pay'].median())
```

Missing Min_Pay: 23302, Missing Max_Pay: 23302

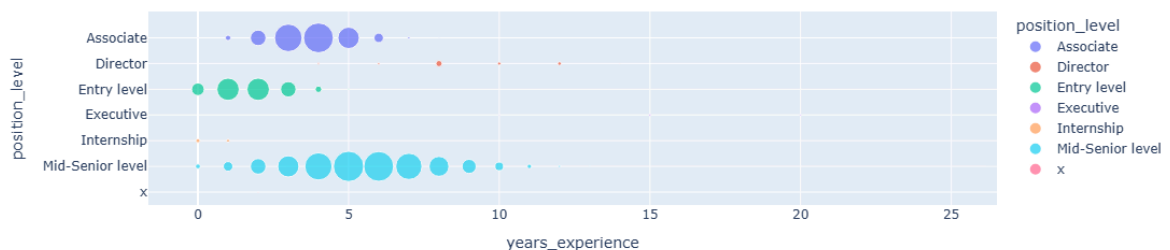
```
In [32]: industry_apps = df.groupby("industry")["Applicants"].sum().reset_index()
fig = px.treemap(industry_apps, path=["industry"], values="Applicants",
                 title="Applications by Industry")
fig.show()
```

Applications by Industry



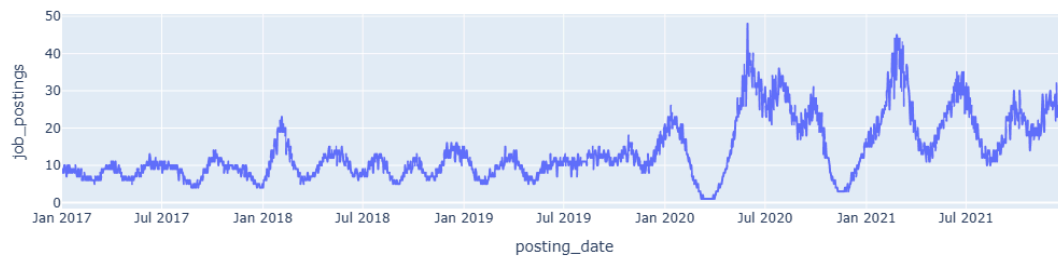
```
In [33]: exp_level = df.groupby(["position_level", "years_experience"]).size().reset_index()
fig = px.scatter(exp_level, x="years_experience", y="position_level", size="Count",
                 color="position_level", title="Experience vs Job Level")
fig.show()
```

Experience vs Job Level



```
In [34]: time_trend = df.groupby("posting_date").size().reset_index(name="job_postings")
fig = px.line(time_trend, x="posting_date", y="job_postings",
               title="Job Postings Trend Over Time")
fig.show()
```

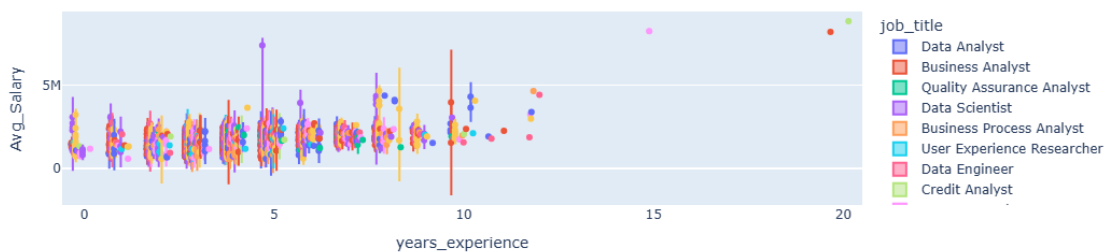
Job Postings Trend Over Time



```
In [36]: df['Avg_Salary'] = (df['min_pay'] + df['max_pay']) / 2

fig = px.violin(df, x="years_experience", y="Avg_Salary", color="job_title",
                box=True, points="all", title="Salary Distribution by Experience")
fig.show()
```

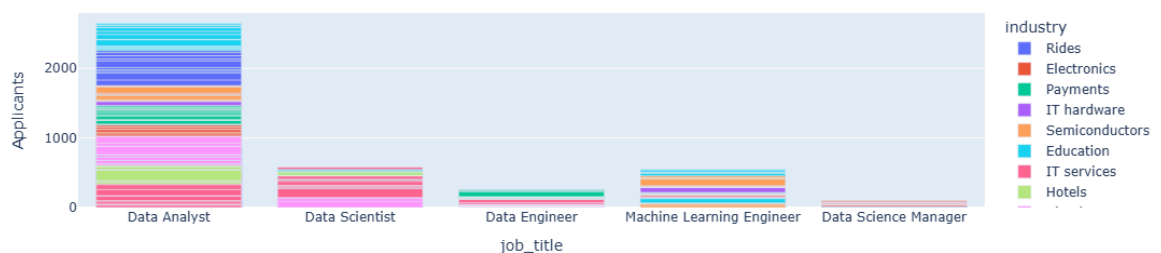
Salary Distribution by Experience



```
In [37]: data_roles = ["Data Scientist", "Data Analyst", "Data Engineer", "Machine Learning"]
df_data = df[df['job_title'].isin(data_roles)]

fig = px.bar(df_data, x="job_title", y="Applicants", color="industry",
            title="Applicants for Data Roles by Industry")
fig.show()
```

Applicants for Data Roles by Industry



```
In [39]: tableau_jobs = df[df['skills'].str.contains("Tableau", case=False, na=False)]
tableau_count = len(tableau_jobs)
tableau_applicants = tableau_jobs['Applicants'].sum()
percentage = (tableau_count / len(df)) * 100

print(f"Tableau mentioned in {percentage:.2f}% of postings, with {tableau_applicants} applicants.")
```

Tableau mentioned in 9.28% of postings, with 40798.0 applicants.

```
In [41]: import plotly.express as px

# Filter postings mentioning Tableau
tableau_jobs = df[df['skills'].str.contains("Tableau", case=False, na=False)]
tableau_count = len(tableau_jobs)
tableau_applicants = tableau_jobs['Applicants'].sum()
percentage = (tableau_count / len(df)) * 100

print(f"Tableau mentioned in {percentage:.2f}% of postings, with {tableau_applicants} applicants.")

# Donut chart for recruiter-friendly visualization
fig = px.pie(
    values=[tableau_count, len(df)-tableau_count],
    names=["Tableau Required", "Other Roles"],
    hole=0.4,
    title="Share of Job Postings Requiring Tableau"
)
fig.update_traces(textinfo="percent+label")
fig.show()
```

Tableau mentioned in 9.28% of postings, with 40798 applicants.

Share of Job Postings Requiring Tableau



Recommendations for Recruiters

1. Industries Hiring Most Action: Build partnerships with the top-hiring sectors (e.g., IT services, consulting, BFSI).

Impact: Ensures candidates are funneled into industries with the highest demand, increasing placement success rates.

2. Experience Trends Action: Many postings are mid-senior level. Encourage candidates to upskill in advanced analytics, cloud, and visualization tools.

Impact: Bridges the gap between entry-level talent and industry expectations, improving job readiness.

3. Salary Insights Action: Highlight competitive pay ranges in analytics roles during career counseling.

Impact: Helps candidates negotiate better and positions your service as a trusted advisor on compensation benchmarks.

4. Tableau Demand Action: Promote Tableau (and complementary tools like Power BI) as must-learn skills.

Impact: Candidates with Tableau expertise stand out in ~10% of postings, where demand is high and applicant volume is significant.

```
In [44]: import pandas as pd
import networkx as nx
import plotly.graph_objects as go

# Ensure skills column is string and drop NaN
df['skills'] = df['skills'].astype(str)

# Split skills into lists
df['skills_list'] = df['skills'].apply(lambda x: [s.strip() for s in x.split(",")])

# Build co-occurrence pairs
pairs = []
for skills in df['skills_list']:
    if len(skills) > 1: # only consider rows with multiple skills
        for i in range(len(skills)):
            for j in range(i+1, len(skills)):
                pairs.append((skills[i], skills[j]))

# Create graph with edge weights
G = nx.Graph()
for a, b in pairs:
    if G.has_edge(a, b):
        G[a][b]['weight'] += 1
    else:
        G.add_edge(a, b, weight=1)

# Focus only on Tableau and its top 10 co-occurring skills
if "Tableau" in G.nodes:
    neighbors = list(G.neighbors("Tableau"))
    neighbor_counts = {n: G["Tableau"][n]['weight'] for n in neighbors}
    top_neighbors = sorted(neighbor_counts, key=neighbor_counts.get, reverse=True)
    sub_nodes = ["Tableau"] + top_neighbors
    H = G.subgraph(sub_nodes)
else:
    H = G

# Position nodes neatly
pos = nx.spring_layout(H, k=0.8, seed=42)

# Build Plotly edges
edge_x, edge_y = [], []
for edge in H.edges():
    x0, y0 = pos[edge[0]]
    x1, y1 = pos[edge[1]]
    edge_x += [x0, x1, None]
    edge_y += [y0, y1, None]

edge_trace = go.Scatter(
    x=edge_x, y=edge_y,
    line=dict(width=2, color='lightgray'),
    hoverinfo='none',
    mode='lines')

# Build Plotly nodes
node_x, node_y, node_text = [], [], []
for node in H.nodes():
```

